

DataForge

Real-World Data Cleaning & Validation Pipeline

Project Workflow & Frozen v1.0 Specification

Purpose: build one compact, commercial-grade public GitHub project that proves the ability to turn messy Excel/CSV data into validated, analysis-ready deliverables with traceability.

Field	Frozen decision
Portfolio role	Commercial data engineering / cleaning / validation proof
Primary audience	Upwork clients, startup hiring managers, junior Data/Python roles
Target build window	Approx. 2-3 focused development days; stop after v1 acceptance
Core stack	Python, pandas, openpyxl, pandera or pydantic, rapidfuzz, pytest, GitHub Actions
Public deliverables	GitHub repository + demo raw data + cleaned outputs + QA report + README
Design principle	Never silently destroy customer data

Portfolio architecture: Kronos demonstrates ML / time-series / evaluation rigor. DataForge demonstrates practical client delivery, data quality, automation, and engineering judgment. These two repositories are sufficient for the initial external portfolio.

1. Project Objective

DataForge simulates a realistic client engagement: a small e-commerce business provides inconsistent monthly Excel/CSV files and asks for one reliable dataset. The repository must show the entire transformation from raw input to validated output, including rejected records and an audit trail.

Success is not measured by algorithmic novelty. Success is measured by whether a non-technical client can understand the result, whether every material transformation is explainable, and whether the pipeline is reproducible and tested.

1.1 User story

"I have several messy sales spreadsheets from different months. Please consolidate them, clean formatting problems and duplicates, identify invalid records, and return one trustworthy Excel/CSV dataset with a clear summary of what changed."

1.2 Explicit non-goals for v1

- No database, data warehouse, Airflow, Spark, cloud deployment, API service, dashboard platform, LLM, or machine-learning model.
- No attempt to build a universal auto-cleaner for arbitrary datasets.
- No hidden mutation of source files; raw demo inputs remain immutable.
- No research-style governance framework or multi-week architecture expansion.
- No production claim beyond the bounded demo schema and documented assumptions.

2. Demo Dataset Design

Create a synthetic but realistic e-commerce dataset. It must be safe to publish and intentionally contain controlled defects. The generator should make the clean ground truth known, then inject defects so tests can verify expected behavior.

Input	Purpose	Examples of injected defects
monthly sales files	Core transactional data	mixed CSV/XLSX, column aliases, duplicate orders, invalid quantity, malformed prices,
customers.xlsx	Customer reference	name casing/spacing, missing IDs, malformed email, near-duplicate names
products.csv	Product reference	category aliases, missing SKU, invalid unit price

Required defect classes

- Schema: Customer ID / customer_id / CustomerID; optional and unexpected columns.
- Locale: EUR values such as '1 299,00 EUR', 'EUR1,299.00', '1299'; mixed decimal/thousands separators.
- Dates: ISO, DD/MM/YYYY, textual dates, invalid dates.
- Categories: France / france / FR / FRA; inconsistent product categories.
- Missingness: critical identifier missing versus optional descriptive field missing.
- Duplicates: exact row duplicates, duplicate order_id, normalized near-duplicate customer names.
- Business-rule violations: quantity <= 0, price < 0, impossible or future date under frozen demo date.
- Text hygiene: leading/trailing whitespace, casing, duplicated internal spaces.

3. Functional Requirements

ID	Requirement
F01 Multi-file ingestion	Read .csv and .xlsx, including selected workbook sheets, from an input directory. Preserve source filename and source
F02 Schema normalization	Map documented aliases to canonical snake_case columns; report missing required and unexpected columns.
F03 Type & locale normalization	Normalize dates, EUR currency, whitespace, case, country codes, email text and categorical aliases.
F04 Missing-data policy	Critical missing identifiers are quarantined; optional missing values are retained; no blanket dropna().
F05 Duplicate handling	Detect exact duplicates and duplicate business keys. Near-duplicate customer matching may be flagged but must not s
F06 Business validation	Apply explicit constraints such as quantity > 0, unit_price >= 0, unique order_id and valid date.
F07 Quarantine	Invalid records are separated into rejected_rows.csv with source, row ID, rule code and human-readable reason.
F08 Audit trail	Material changes are recorded with source row, field, original value, cleaned value and rule/action code.
F09 Quality report	Produce a concise HTML report with row counts, accepted/rejected counts, defect counts and quality metrics.
F10 Client outputs	Produce cleaned_sales.xlsx, cleaned_sales.csv, rejected_rows.csv, audit_log.csv and cleaning_summary.json.

4. Data Policies

Condition	Action	Rationale
Recoverable formatting defect	Normalize + audit	Value is semantically recoverable
Critical ID missing	Quarantine	Cannot safely identify business entity
Invalid numeric business value	Quarantine	Do not invent commercial facts
Optional field missing	Retain null	Avoid unnecessary row loss
Exact duplicate	Deduplicate + audit	Deterministic
Ambiguous fuzzy duplicate	Flag only	No unsafe automatic merge
Unknown category	Retain/flag per schema	Avoid forced misclassification

Frozen principle

Never silently destroy customer data. Every dropped, quarantined, deduplicated, normalized, or materially changed record must be explainable from machine-readable evidence.

5. Repository Architecture

```
dataforge/ README.md pyproject.toml src/dataforge/ cli.py ingest.py schema.py normalize.py dedupe.py
validate.py audit.py report.py pipeline.py data/demo_raw/ data/demo_expected/ examples/output/ tests/
test_ingest.py test_normalize.py test_dedupe.py test_validate.py test_pipeline.py scripts/
generate_demo_data.py .github/workflows/tests.yml LICENSE
```

The public demo command should be simple enough for a client to understand:

```
python -m dataforge.cli --input data/demo_raw --output examples/output
```

6. Development Workflow

Phase	Work	Exit criterion
DF-000 Freeze	Commit this specification, canonical schema, rules, outputs, and non-goals.	All unresolved core requirement.
DF-001 Demo data	Build deterministic synthetic ground truth and defect injector.	All required defect classes represented and reproducible.

Phase	Work	Exit criterion
DF-002 Ingestion/schema	CSV/XLSX ingestion, provenance columns, alias mapping, schema checks	Ingestion tests pass; malformed schema fails clearly.
DF-003 Normalization/dedup	Dates, currency, country/category, text, email hygiene.	Known recoverable defects normalize to expected values.
DF-004 Validation/duplicate	Business rules, exact/key duplicates, fuzzy flags, quarantine.	No invalid record enters accepted output under frozen rules.
DF-005 Audit/report	Audit log, summary JSON, HTML QA report.	Every material action traceable; counts reconcile.
DF-006 End-to-end	CLI, Excel/CSV outputs, integration tests, deterministic demo run.	One command reproduces all expected deliverables.
DF-007 Portfolio portfolio	README, before/after example, architecture diagram, screenshots, GitHub Actions	Non-technical reviewer understands value in <=10 seconds.
DF-008 Release gate	Run acceptance suite and scope review; tag v1.0.	PASS or explicit BLOCKED; no scope creep workaround.

7. Acceptance Gates

G1 Reproducibility - Fresh environment + documented install + one command produces expected demo outputs.

G2 Reconciliation - $\text{input_rows} = \text{accepted_rows} + \text{quarantined_rows} + \text{explicitly deduplicated_rows}$, with documented accounting semantics.

G3 No silent loss - Every excluded row has a deterministic reason and source provenance.

G4 Validation integrity - Rows violating frozen hard business rules cannot appear in cleaned output.

G5 Audit integrity - Every material normalization/deduplication/quarantine event has an audit record.

G6 Test quality - Unit + integration tests pass in GitHub Actions; defect fixtures cover all frozen classes.

G7 Client usability - cleaned XLSX/CSV opens correctly; report is readable; filenames and outputs are self-explanatory.

G8 Portfolio clarity - README opening shows problem, before/after, result metrics, command, outputs, and limitations without requiring code inspection.

Release decision

Result	Action
PASS	Tag v1.0, pin repository on GitHub, use in Upwork/Wellfound applications.
BLOCKED	Fix only the failed frozen requirement; do not add unrelated features.
REWORK	Allowed only if a core assumption is proven invalid; document why before changing scope.

8. Scope Guard / Stop Conditions

DataForge exists to become a portfolio asset quickly. The following requests are automatically deferred to a future version unless required to pass a frozen v1 gate:

- Database or warehouse integration.
- Cloud hosting, web application, authentication or user accounts.
- Streaming ingestion or distributed processing.
- Spark, Airflow, Kafka, dbt or Kubernetes.
- LLM-based cleaning, anomaly ML or predictive modeling.
- Generic plugin architecture for arbitrary schemas.
- Interactive BI dashboard.

- Enterprise observability, distributed tracing or complex governance.

Hard stop: once G1-G8 pass and README is portfolio-ready, v1 is complete. Additional ideas go to BACKLOG.md; they do not delay publication.

9. README / External Presentation Specification

The first screen of README must communicate commercial value before implementation detail.

Order	Required content
1	Title + one-sentence value proposition: messy spreadsheets -> validated analysis-ready data.
2	Before/After mini-table using 3-5 representative records.
3	Headline demo metrics: files, rows, accepted, quarantined, duplicates, normalized fields.
4	One-command usage example.
5	Output files and screenshot/link to QA report.
6	Key safeguards: provenance, quarantine, audit trail, no silent deletion.
7	Architecture / pipeline diagram.
8	Tests, limitations, installation, detailed rules.

Suggested README headline

From messy spreadsheets to validated, analysis-ready data.

DataForge is a reproducible Python pipeline that cleans, validates and consolidates messy Excel and CSV datasets while preserving source provenance and a complete audit trail.

10. Deliverable Contract

Artifact	Required
cleaned_sales.xlsx	Yes - client-friendly workbook
cleaned_sales.csv	Yes - machine-friendly output
rejected_rows.csv	Yes - invalid/quarantined records with reasons
audit_log.csv	Yes - material transformations and actions
cleaning_summary.json	Yes - machine-readable reconciliation and metrics
data_quality_report.html	Yes - human-readable QA report
demo raw files	Yes - synthetic, publishable
tests + CI	Yes
README screenshots / before-after	Yes

11. Suggested Task Prompt for Coding Agent

Implement DataForge strictly against the frozen v1.0 specification. Priorities: 1. correctness and deterministic reconciliation; 2. provenance and no silent data loss; 3. clear client-facing outputs; 4. tests before portfolio polish. Do not add databases, cloud services, Spark, Airflow, LLMs, dashboards, or unrelated abstractions. If a frozen requirement is ambiguous or internally inconsistent, stop and report BLOCKED with: - exact ambiguity, - affected requirement/gate, - minimum proposed resolution, - files that would change. Do not silently reinterpret the specification. At the

end of each DF phase, run relevant tests and report: - completed requirements, - test results, - artifacts produced, - unresolved blockers, - next authorized phase.

12. Final Definition of Done

- A fresh user can clone, install and run the demo using documented commands.
- The synthetic raw inputs visibly contain realistic spreadsheet defects.
- The cleaned output is deterministic and satisfies all frozen hard rules.
- Rejected/deduplicated records reconcile with the input and are explainable.
- The audit log preserves source provenance and material changes.
- The QA report gives a non-technical client a useful summary without reading code.
- GitHub Actions passes.
- README communicates value within the first screen and includes a strong before/after demonstration.
- No v1 non-goal has been added merely to make the repository look more sophisticated.
- Repository is tagged v1.0 and ready to pin beside Kronos.

At that point, stop development and use the project externally. The portfolio objective has been achieved.